

## 3. Classes and Inheritance

As an object-oriented programming language, Python supports a full range of features, such as inheritance, polymorphism, and encapsulation. Getting things done in Python often requires writing new classes and defining how they interact through their interfaces and hierarchies.

Python's classes and inheritance make it easy to express your program's intended behaviors with objects. They allow you to improve and expand functionality over time. They provide flexibility in an environment of changing requirements. Knowing how to use them well enables you to write maintainable code.

### Item 22: Prefer Helper Classes Over Bookkeeping with Dictionaries and Tuples

Python's built-in dictionary type is wonderful for maintaining dynamic internal state over the lifetime of an object. By *dynamic*, I mean situations in which you need to do bookkeeping for an unexpected set of identifiers. For example, say you want to record the grades of a set of students whose names aren't known in advance. You can define a class to store the names in a dictionary instead of using a predefined attribute for each student.

[Click here to view code image](#)

```
class SimpleGradebook(object):
    def __init__(self):
        self._grades = {}

    def add_student(self, name):
        self._grades[name] = []

    def report_grade(self, name, score):
        self._grades[name].append(score)

    def average_grade(self, name):
        grades = self._grades[name]
        return sum(grades) / len(grades)
```

Using the class is simple.

[Click here to view code image](#)

```
book = SimpleGradebook()
book.add_student('Isaac Newton')
book.report_grade('Isaac Newton', 90)
# ...
print(book.average_grade('Isaac Newton'))

>>>
90.0
```

Dictionaries are so easy to use that there's a danger of overextending them to write brittle code. For example, say you want to extend the `SimpleGradebook` class to keep a list of grades by subject, not just overall. You can do this by changing the `_grades` dictionary to map student names (the keys) to yet another dictionary (the values). The

innermost dictionary will map subjects (the keys) to grades (the values).

[Click here to view code image](#)

```
class BySubjectGradebook(object):
    def __init__(self):
        self._grades = {}
    def add_student(self, name):
        self._grades[name] = {}
```

This seems straightforward enough. The `report_grade` and `average_grade` methods will gain quite a bit of complexity to deal with the multilevel dictionary, but it's manageable.

[Click here to view code image](#)

```
    def report_grade(self, name, subject, grade):
        by_subject = self._grades[name]
        grade_list = by_subject.setdefault(subject, [])
        grade_list.append(grade)

    def average_grade(self, name):
        by_subject = self._grades[name]
        total, count = 0, 0
        for grades in by_subject.values():
            total += sum(grades)
            count += len(grades)
        return total / count
```

Using the class remains simple.

[Click here to view code image](#)

```
book = BySubjectGradebook()
book.add_student('Albert Einstein')
book.report_grade('Albert Einstein', 'Math', 75)
book.report_grade('Albert Einstein', 'Math', 65)
book.report_grade('Albert Einstein', 'Gym', 90)
book.report_grade('Albert Einstein', 'Gym', 95)
```

Now, imagine your requirements change again. You also want to track the weight of each score toward the overall grade in the class so midterms and finals are more important than pop quizzes. One way to implement this feature is to change the innermost dictionary; instead of mapping subjects (the keys) to grades (the values), I can use the tuple `(score, weight)` as values.

[Click here to view code image](#)

```
class WeightedGradebook(object):
    # ...
    def report_grade(self, name, subject, score, weight):
        by_subject = self._grades[name]
        grade_list = by_subject.setdefault(subject, [])
        grade_list.append((score, weight))
```

Although the changes to `report_grade` seem simple—just make the value a tuple—the `average_grade` method now has a loop within a loop and is difficult to read.

[Click here to view code image](#)

```
    def average_grade(self, name):
        by_subject = self._grades[name]
```

```

score_sum, score_count = 0, 0
for subject, scores in by_subject.items():
    subject_avg, total_weight = 0, 0
    for score, weight in scores:
        # ...
    return score_sum / score_count

```

Using the class has also gotten more difficult. It's unclear what all of the numbers in the positional arguments mean.

[Click here to view code image](#)

```
book.report_grade('Albert Einstein', 'Math', 80, 0.10)
```

When you see complexity like this happen, it's time to make the leap from dictionaries and tuples to a hierarchy of classes.

At first, you didn't know you'd need to support weighted grades, so the complexity of additional helper classes seemed unwarranted. Python's built-in dictionary and tuple types made it easy to keep going, adding layer after layer to the internal bookkeeping. But you should avoid doing this for more than one level of nesting (i.e., avoid dictionaries that contain dictionaries). It makes your code hard to read by other programmers and sets you up for a maintenance nightmare.

As soon as you realize the bookkeeping is getting complicated, break it all out into classes. This lets you provide well-defined interfaces that better encapsulate your data. This also enables you to create a layer of abstraction between your interfaces and your concrete implementations.

## Refactoring to Classes

You can start moving to classes at the bottom of the dependency tree: a single grade. A class seems too heavyweight for such simple information. A tuple, though, seems appropriate because grades are immutable. Here, I use the tuple (score, weight) to track grades in a list:

[Click here to view code image](#)

```

grades = []
grades.append((95, 0.45))
# ...
total = sum(score * weight for score, weight in grades)
total_weight = sum(weight for _, weight in grades)
average_grade = total / total_weight

```

The problem is that plain tuples are positional. When you want to associate more information with a grade, like a set of notes from the teacher, you'll need to rewrite every usage of the two-tuple to be aware that there are now three items present instead of two. Here, I use `_` (the underscore variable name, a Python convention for unused variables) to capture the third entry in the tuple and just ignore it:

[Click here to view code image](#)

```

grades = []
grades.append((95, 0.45, 'Great job'))
# ...
total = sum(score * weight for score, weight, _ in grades)

```

```
total_weight = sum(weight for _, weight, _ in grades)
average_grade = total / total_weight
```

This pattern of extending tuples longer and longer is similar to deepening layers of dictionaries. As soon as you find yourself going longer than a two-tuple, it's time to consider another approach.

The `namedtuple` type in the `collections` module does exactly what you need. It lets you easily define tiny, immutable data classes.

[Click here to view code image](#)

```
import collections
Grade = collections.namedtuple('Grade', ('score', 'weight'))
```

These classes can be constructed with positional or keyword arguments. The fields are accessible with named attributes. Having named attributes makes it easy to move from a `namedtuple` to your own class later if your requirements change again and you need to add behaviors to the simple data containers.

---

## Limitations of `namedtuple`

Although useful in many circumstances, it's important to understand when `namedtuple` can cause more harm than good.

- You can't specify default argument values for `namedtuple` classes. This makes them unwieldy when your data may have many optional properties. If you find yourself using more than a handful of attributes, defining your own class may be a better choice.
- The attribute values of `namedtuple` instances are still accessible using numerical indexes and iteration. Especially in externalized APIs, this can lead to unintentional usage that makes it harder to move to a real class later. If you're not in control of all of the usage of your `namedtuple` instances, it's better to define your own class.

---

Next, you can write a class to represent a single subject that contains a set of grades.

[Click here to view code image](#)

```
class Subject(object):
    def __init__(self):
        self._grades = []

    def report_grade(self, score, weight):
        self._grades.append(Grade(score, weight))

    def average_grade(self):
        total, total_weight = 0, 0
        for grade in self._grades:
            total += grade.score * grade.weight
            total_weight += grade.weight
        return total / total_weight
```

Then you would write a class to represent a set of subjects that are being studied by a single student.

[Click here to view code image](#)

```

class Student(object):
    def __init__(self):
        self._subjects = {}

    def subject(self, name):
        if name not in self._subjects:
            self._subjects[name] = Subject()
        return self._subjects[name]

    def average_grade(self):
        total, count = 0, 0
        for subject in self._subjects.values():
            total += subject.average_grade()
            count += 1
        return total / count

```

Finally, you'd write a container for all of the students keyed dynamically by their names.

[Click here to view code image](#)

```

class Gradebook(object):
    def __init__(self):
        self._students = {}

    def student(self, name):
        if name not in self._students:
            self._students[name] = Student()
        return self._students[name]

```

The line count of these classes is almost double the previous implementation's size. But this code is much easier to read. The example driving the classes is also more clear and extensible.

[Click here to view code image](#)

```

book = Gradebook()
albert = book.student('Albert Einstein')
math = albert.subject('Math')
math.report_grade(80, 0.10)
# ...
print(albert.average_grade())

>>>
81.5

```

If necessary, you can write backwards-compatible methods to help migrate usage of the old API style to the new hierarchy of objects.

## Things to Remember

- ◆ Avoid making dictionaries with values that are other dictionaries or long tuples.
- ◆ Use `namedtuple` for lightweight, immutable data containers before you need the flexibility of a full class.
- ◆ Move your bookkeeping code to use multiple helper classes when your internal state dictionaries get complicated.

## Item 23: Accept Functions for Simple Interfaces Instead of Classes

Many of Python's built-in APIs allow you to customize behavior by passing in a function. These *hooks* are used by APIs to call back your code while they execute. For example, the `list` type's `sort` method takes an optional `key` argument that's used to determine each index's value for sorting. Here, I sort a list of names based on their lengths by providing a `lambda` expression as the `key` hook:

[Click here to view code image](#)

```
names = ['Socrates', 'Archimedes', 'Plato', 'Aristotle']
names.sort(key=lambda x: len(x))
print(names)

>>>
['Plato', 'Socrates', 'Aristotle', 'Archimedes']
```

In other languages, you might expect hooks to be defined by an abstract class. In Python, many hooks are just stateless functions with well-defined arguments and return values. Functions are ideal for hooks because they are easier to describe and simpler to define than classes. Functions work as hooks because Python has *first-class* functions: Functions and methods can be passed around and referenced like any other value in the language.

For example, say you want to customize the behavior of the `defaultdict` class (see [Item 46: "Use Built-in Algorithms and Data Structures"](#) for details). This data structure allows you to supply a function that will be called each time a missing key is accessed. The function must return the default value the missing key should have in the dictionary. Here, I define a hook that logs each time a key is missing and returns `0` for the default value:

```
def log_missing():
    print('Key added')
    return 0
```

Given an initial dictionary and a set of desired increments, I can cause the `log_missing` function to run and print twice (for `'red'` and `'orange'`).

[Click here to view code image](#)

```
current = {'green': 12, 'blue': 3}
increments = [
    ('red', 5),
    ('blue', 17),
    ('orange', 9),
]
result = defaultdict(log_missing, current)
print('Before:', dict(result))
for key, amount in increments:
    result[key] += amount
print('After: ', dict(result))

>>>
Before: {'green': 12, 'blue': 3}
Key added
Key added
```

After: {'orange': 9, 'green': 12, 'blue': 20, 'red': 5}

Supplying functions like `log_missing` makes APIs easy to build and test because it separates side effects from deterministic behavior. For example, say you now want the default value hook passed to `defaultdict` to count the total number of keys that were missing. One way to achieve this is using a stateful closure (see [Item 15: “Know How Closures Interact with Variable Scope”](#) for details). Here, I define a helper function that uses such a closure as the default value hook:

[Click here to view code image](#)

```
def increment_with_report(current, increments):
    added_count = 0

    def missing():
        nonlocal added_count # Stateful closure
        added_count += 1
        return 0

    result = defaultdict(missing, current)
    for key, amount in increments:
        result[key] += amount

    return result, added_count
```

Running this function produces the expected result (2), even though the `defaultdict` has no idea that the `missing` hook maintains state. This is another benefit of accepting simple functions for interfaces. It's easy to add functionality later by hiding state in a closure.

[Click here to view code image](#)

```
result, count = increment_with_report(current, increments)
assert count == 2
```

The problem with defining a closure for stateful hooks is that it's harder to read than the stateless function example. Another approach is to define a small class that encapsulates the state you want to track.

```
class CountMissing(object):
    def __init__(self):
        self.added = 0

    def missing(self):
        self.added += 1
        return 0
```

In other languages, you might expect that now `defaultdict` would have to be modified to accommodate the interface of `CountMissing`. But in Python, thanks to first-class functions, you can reference the `CountMissing.missing` method directly on an object and pass it to `defaultdict` as the default value hook. It's trivial to have a method satisfy a function interface.

[Click here to view code image](#)

```
counter = CountMissing()
result = defaultdict(counter.missing, current) # Method ref

for key, amount in increments:
```

```
    result[key] += amount
    assert counter.added == 2
```

Using a helper class like this to provide the behavior of a stateful closure is clearer than the `increment_with_report` function above. However, in isolation it's still not immediately obvious what the purpose of the `CountMissing` class is. Who constructs a `CountMissing` object? Who calls the `missing` method? Will the class need other public methods to be added in the future? Until you see its usage with `defaultdict`, the class is a mystery.

To clarify this situation, Python allows classes to define the `__call__` special method. `__call__` allows an object to be called just like a function. It also causes the `callable` built-in function to return `True` for such an instance.

[Click here to view code image](#)

```
class BetterCountMissing(object):
    def __init__(self):
        self.added = 0

    def __call__(self):
        self.added += 1
        return 0

counter = BetterCountMissing()
counter()
assert callable(counter)
```

Here, I use a `BetterCountMissing` instance as the default value hook for a `defaultdict` to track the number of missing keys that were added:

[Click here to view code image](#)

```
counter = BetterCountMissing()
result = defaultdict(counter, current) # Relies on __call__
for key, amount in increments:
    result[key] += amount
assert counter.added == 2
```

This is much clearer than the `CountMissing.missing` example. The `__call__` method indicates that a class's instances will be used somewhere a function argument would also be suitable (like API hooks). It directs new readers of the code to the entry point that's responsible for the class's primary behavior. It provides a strong hint that the goal of the class is to act as a stateful closure.

Best of all, `defaultdict` still has no view into what's going on when you use `__call__`. All that `defaultdict` requires is a function for the default value hook. Python provides many different ways to satisfy a simple function interface depending on what you need to accomplish.

## Things to Remember

- ◆ Instead of defining and instantiating classes, functions are often all you need for simple interfaces between components in Python.
- ◆ References to functions and methods in Python are first class, meaning they can be

used in expressions like any other type.

- ◆ The `__call__` special method enables instances of a class to be called like plain Python functions.
- ◆ When you need a function to maintain state, consider defining a class that provides the `__call__` method instead of defining a stateful closure (see [Item 15: “Know How Closures Interact with Variable Scope”](#)).

## Item 24: Use `@classmethod` Polymorphism to Construct Objects Generically

In Python, not only do the objects support polymorphism, but the classes do as well. What does that mean, and what is it good for?

Polymorphism is a way for multiple classes in a hierarchy to implement their own unique versions of a method. This allows many classes to fulfill the same interface or abstract base class while providing different functionality (see [Item 28: “Inherit from `collections.abc` for Custom Container Types”](#) for an example).

For example, say you’re writing a MapReduce implementation and you want a common class to represent the input data. Here, I define such a class with a `read` method that must be defined by subclasses:

[Click here to view code image](#)

```
class InputData(object):
    def read(self):
        raise NotImplementedError
```

Here, I have a concrete subclass of `InputData` that reads data from a file on disk:

[Click here to view code image](#)

```
class PathInputData(InputData):
    def __init__(self, path):
        super().__init__()
        self.path = path

    def read(self):
        return open(self.path).read()
```

You could have any number of `InputData` subclasses like `PathInputData` and each of them could implement the standard interface for `read` to return the bytes of data to process. Other `InputData` subclasses could read from the network, decompress data transparently, etc.

You’d want a similar abstract interface for the MapReduce worker that consumes the input data in a standard way.

[Click here to view code image](#)

```
class Worker(object):
    def __init__(self, input_data):
        self.input_data = input_data
        self.result = None
```

```
def map(self):
    raise NotImplementedError

def reduce(self, other):
    raise NotImplementedError
```

Here, I define a concrete subclass of `Worker` to implement the specific MapReduce function I want to apply: a simple newline counter:

[Click here to view code image](#)

```
class LineCountWorker(Worker):
    def map(self):
        data = self.input_data.read()
        self.result = data.count('\n')

    def reduce(self, other):
        self.result += other.result
```

It may look like this implementation is going great, but I’ve reached the biggest hurdle in all of this. What connects all of these pieces? I have a nice set of classes with reasonable interfaces and abstractions—but that’s only useful once the objects are constructed. What’s responsible for building the objects and orchestrating the MapReduce?

The simplest approach is to manually build and connect the objects with some helper functions. Here, I list the contents of a directory and construct a `PathInputData` instance for each file it contains:

[Click here to view code image](#)

```
def generate_inputs(data_dir):
    for name in os.listdir(data_dir):
        yield PathInputData(os.path.join(data_dir, name))
```

Next, I create the `LineCountWorker` instances using the `InputData` instances returned by `generate_inputs`.

[Click here to view code image](#)

```
def create_workers(input_list):
    workers = []
    for input_data in input_list:
        workers.append(LineCountWorker(input_data))
    return workers
```

I execute these `Worker` instances by fanning out the `map` step to multiple threads (see [Item 37: “Use Threads for Blocking I/O, Avoid for Parallelism”](#)). Then, I call `reduce` repeatedly to combine the results into one final value.

[Click here to view code image](#)

```
def execute(workers):
    threads = [Thread(target=w.map) for w in workers]
    for thread in threads: thread.start()
    for thread in threads: thread.join()

    first, rest = workers[0], workers[1:]
    for worker in rest:
        first.reduce(worker)
    return first.result
```

Finally, I connect all of the pieces together in a function to run each step.

[Click here to view code image](#)

```
def mapreduce(data_dir):
    inputs = generate_inputs(data_dir)
    workers = create_workers(inputs)
    return execute(workers)
```

Running this function on a set of test input files works great.

[Click here to view code image](#)

```
from tempfile import TemporaryDirectory

def write_test_files(tmpdir):
    # ...

with TemporaryDirectory() as tmpdir:
    write_test_files(tmpdir)
    result = mapreduce(tmpdir)

print('There are', result, 'lines')

>>>
There are 4360 lines
```

What's the problem? The huge issue is the `mapreduce` function is not generic at all. If you want to write another `InputData` or `Worker` subclass, you would also have to rewrite the `generate_inputs`, `create_workers`, and `mapreduce` functions to match.

This problem boils down to needing a generic way to construct objects. In other languages, you'd solve this problem with constructor polymorphism, requiring that each `InputData` subclass provides a special constructor that can be used generically by the helper methods that orchestrate the MapReduce. The trouble is that Python only allows for the single constructor method `__init__`. It's unreasonable to require every `InputData` subclass to have a compatible constructor.

The best way to solve this problem is with `@classmethod` polymorphism. This is exactly like the instance method polymorphism I used for `InputData.read`, except that it applies to whole classes instead of their constructed objects.

Let me apply this idea to the MapReduce classes. Here, I extend the `InputData` class with a generic class method that's responsible for creating new `InputData` instances using a common interface:

[Click here to view code image](#)

```
class GenericInputData(object):
    def read(self):
        raise NotImplementedError

    @classmethod
    def generate_inputs(cls, config):
        raise NotImplementedError
```

I have `generate_inputs` take a dictionary with a set of configuration parameters that are up to the `InputData` concrete subclass to interpret. Here, I use the `config` to find

the directory to list for input files:

[Click here to view code image](#)

```
class PathInputData(GenericInputData):
    # ...
    def read(self):
        return open(self.path).read()

    @classmethod
    def generate_inputs(cls, config):
        data_dir = config['data_dir']
        for name in os.listdir(data_dir):
            yield cls(os.path.join(data_dir, name))
```

Similarly, I can make the `create_workers` helper part of the `GenericWorker` class. Here, I use the `input_class` parameter, which must be a subclass of `GenericInputData`, to generate the necessary inputs. I construct instances of the `GenericWorker` concrete subclass using `cls()` as a generic constructor.

[Click here to view code image](#)

```
class GenericWorker(object):
    # ...
    def map(self):
        raise NotImplementedError

    def reduce(self, other):
        raise NotImplementedError

    @classmethod
    def create_workers(cls, input_class, config):
        workers = []
        for input_data in input_class.generate_inputs(config):
            workers.append(cls(input_data))
        return workers
```

Note that the call to `input_class.generate_inputs` above is the class polymorphism I'm trying to show. You can also see how `create_workers` calling `cls` provides an alternate way to construct `GenericWorker` objects besides using the `__init__` method directly.

The effect on my concrete `GenericWorker` subclass is nothing more than changing its parent class.

[Click here to view code image](#)

```
class LineCountWorker(GenericWorker):
    # ...
```

And finally, I can rewrite the `mapreduce` function to be completely generic.

[Click here to view code image](#)

```
def mapreduce(worker_class, input_class, config):
    workers = worker_class.create_workers(input_class, config)
    return execute(workers)
```

Running the new worker on a set of test files produces the same result as the old implementation. The difference is that the `mapreduce` function requires more parameters so that it can operate generically.

[Click here to view code image](#)

```
with TemporaryDirectory() as tmpdir:
    write_test_files(tmpdir)
    config = {'data_dir': tmpdir}
    result = mapreduce(LineCountWorker, PathInputData, config)
```

Now you can write other `GenericInputData` and `GenericWorker` classes as you wish and not have to rewrite any of the glue code.

## Things to Remember

- ◆ Python only supports a single constructor per class, the `__init__` method.
- ◆ Use `@classmethod` to define alternative constructors for your classes.
- ◆ Use class method polymorphism to provide generic ways to build and connect concrete subclasses.

## Item 25: Initialize Parent Classes with `super`

The old way to initialize a parent class from a child class is to directly call the parent class's `__init__` method with the child instance.

[Click here to view code image](#)

```
class MyBaseClass(object):
    def __init__(self, value):
        self.value = value

class MyChildClass(MyBaseClass):
    def __init__(self):
        MyBaseClass.__init__(self, 5)
```

This approach works fine for simple hierarchies but breaks down in many cases.

If your class is affected by multiple inheritance (something to avoid in general; see [Item 26: “Use Multiple Inheritance Only for Mix-in Utility Classes”](#)), calling the superclasses' `__init__` methods directly can lead to unpredictable behavior.

One problem is that the `__init__` call order isn't specified across all subclasses. For example, here I define two parent classes that operate on the instance's `value` field:

```
class TimesTwo(object):
    def __init__(self):
        self.value *= 2

class PlusFive(object):
    def __init__(self):
        self.value += 5
```

This class defines its parent classes in one ordering.

[Click here to view code image](#)

```
class OneWay(MyBaseClass, TimesTwo, PlusFive):
    def __init__(self, value):
        MyBaseClass.__init__(self, value)
        TimesTwo.__init__(self)
        PlusFive.__init__(self)
```

And constructing it produces a result that matches the parent class ordering.

[Click here to view code image](#)

```
foo = OneWay(5)
print('First ordering is (5 * 2) + 5 =', foo.value)

>>>
First ordering is (5 * 2) + 5 = 15
```

Here's another class that defines the same parent classes but in a different ordering:

[Click here to view code image](#)

```
class AnotherWay(MyBaseClass, PlusFive, TimesTwo):
    def __init__(self, value):
        MyBaseClass.__init__(self, value)
        TimesTwo.__init__(self)
        PlusFive.__init__(self)
```

However, I left the calls to the parent class constructors `PlusFive.__init__` and `TimesTwo.__init__` in the same order as before, causing this class's behavior not to match the order of the parent classes in its definition.

[Click here to view code image](#)

```
bar = AnotherWay(5)
print('Second ordering still is', bar.value)

>>>
Second ordering still is 15
```

Another problem occurs with diamond inheritance. Diamond inheritance happens when a subclass inherits from two separate classes that have the same superclass somewhere in the hierarchy. Diamond inheritance causes the common superclass's `__init__` method to run multiple times, causing unexpected behavior. For example, here I define two child classes that inherit from `MyBaseClass`.

[Click here to view code image](#)

```
class TimesFive(MyBaseClass):
    def __init__(self, value):
        MyBaseClass.__init__(self, value)
        self.value *= 5

class PlusTwo(MyBaseClass):
    def __init__(self, value):
        MyBaseClass.__init__(self, value)
        self.value += 2
```

Then, I define a child class that inherits from both of these classes, making `MyBaseClass` the top of the diamond.

[Click here to view code image](#)

```
class ThisWay(TimesFive, PlusTwo):
    def __init__(self, value):
        TimesFive.__init__(self, value)
        PlusTwo.__init__(self, value)

foo = ThisWay(5)
print('Should be (5 * 5) + 2 = 27 but is', foo.value)
```

```
>>>
```

```
Should be (5 * 5) + 2 = 27 but is 7
```

The output should be 27 because  $(5 * 5) + 2 = 27$ . But the call to the second parent class's constructor, `PlusTwo.__init__`, causes `self.value` to be reset back to 5 when `MyBaseClass.__init__` gets called a second time.

To solve these problems, Python 2.2 added the `super` built-in function and defined the method resolution order (MRO). The MRO standardizes which superclasses are initialized before others (e.g., depth-first, left-to-right). It also ensures that common superclasses in diamond hierarchies are only run once.

Here, I create a diamond-shaped class hierarchy again, but this time I use `super` (in the Python 2 style) to initialize the parent class:

[Click here to view code image](#)

```
# Python 2
class TimesFiveCorrect(MyBaseClass):
    def __init__(self, value):
        super(TimesFiveCorrect, self).__init__(value)
        self.value *= 5

class PlusTwoCorrect(MyBaseClass):
    def __init__(self, value):
        super(PlusTwoCorrect, self).__init__(value)
        self.value += 2
```

Now the top part of the diamond, `MyBaseClass.__init__`, is only run a single time. The other parent classes are run in the order specified in the `class` statement.

[Click here to view code image](#)

```
# Python 2
class GoodWay(TimesFiveCorrect, PlusTwoCorrect):
    def __init__(self, value):
        super(GoodWay, self).__init__(value)

foo = GoodWay(5)
print 'Should be 5 * (5 + 2) = 35 and is', foo.value
```

```
>>>
```

```
Should be 5 * (5 + 2) = 35 and is 35
```

This order may seem backwards at first. Shouldn't `TimesFiveCorrect.__init__` have run first? Shouldn't the result be  $(5 * 5) + 2 = 27$ ? The answer is no. This ordering matches what the MRO defines for this class. The MRO ordering is available on a class method called `mro`.

[Click here to view code image](#)

```
from pprint import pprint
pprint(GoodWay.mro())

>>>
[<class '__main__.GoodWay'>,
 <class '__main__.TimesFiveCorrect'>,
 <class '__main__.PlusTwoCorrect'>,
 <class '__main__.MyBaseClass'>,
```

```
<class 'object'>]
```

When I call `GoodWay(5)`, it in turn calls `TimesFiveCorrect.__init__`, which calls `PlusTwoCorrect.__init__`, which calls `MyBaseClass.__init__`. Once this reaches the top of the diamond, then all of the initialization methods actually do their work in the opposite order from how their `__init__` functions were called.

`MyBaseClass.__init__` assigns the value to 5. `PlusTwoCorrect.__init__` adds 2 to make value equal 7. `TimesFiveCorrect.__init__` multiplies it by 5 to make value equal 35.

The `super` built-in function works well, but it still has two noticeable problems in Python 2:

- Its syntax is a bit verbose. You have to specify the class you're in, the `self` object, the method name (usually `__init__`), and all the arguments. This construction can be confusing to new Python programmers.
- You have to specify the current class by name in the call to `super`. If you ever change the class's name—a very common activity when improving a class hierarchy—you also need to update every call to `super`.

Thankfully, Python 3 fixes these issues by making calls to `super` with no arguments equivalent to calling `super` with `__class__` and `self` specified. In Python 3, you should always use `super` because it's clear, concise, and always does the right thing.

[Click here to view code image](#)

```
class Explicit(MyBaseClass):
    def __init__(self, value):
        super(__class__, self).__init__(value * 2)

class Implicit(MyBaseClass):
    def __init__(self, value):
        super().__init__(value * 2)

assert Explicit(10).value == Implicit(10).value
```

This works because Python 3 lets you reliably reference the current class in methods using the `__class__` variable. This doesn't work in Python 2 because `__class__` isn't defined. You may guess that you could use `self.__class__` as an argument to `super`, but this breaks because of the way `super` is implemented in Python 2.

## Things to Remember

- ◆ Python's standard method resolution order (MRO) solves the problems of superclass initialization order and diamond inheritance.
- ◆ Always use the `super` built-in function to initialize parent classes.

## Item 26: Use Multiple Inheritance Only for Mix-in Utility Classes

Python is an object-oriented language with built-in facilities for making multiple inheritance tractable (see [Item 25: “Initialize Parent Classes with `super`”](#)). However, it’s better to avoid multiple inheritance altogether.

If you find yourself desiring the convenience and encapsulation that comes with multiple inheritance, consider writing a *mix-in* instead. A mix-in is a small class that only defines a set of additional methods that a class should provide. Mix-in classes don’t define their own instance attributes nor require their `__init__` constructor to be called.

Writing mix-ins is easy because Python makes it trivial to inspect the current state of any object regardless of its type. Dynamic inspection lets you write generic functionality a single time, in a mix-in, that can be applied to many other classes. Mix-ins can be composed and layered to minimize repetitive code and maximize reuse.

For example, say you want the ability to convert a Python object from its in-memory representation to a dictionary that’s ready for serialization. Why not write this functionality generically so you can use it with all of your classes?

Here, I define an example mix-in that accomplishes this with a new public method that’s added to any class that inherits from it:

[Click here to view code image](#)

```
class ToDictMixin(object):
    def to_dict(self):
        return self._traverse_dict(self.__dict__)
```

The implementation details are straightforward and rely on dynamic attribute access using `hasattr`, dynamic type inspection with `isinstance`, and accessing the instance dictionary `__dict__`.

[Click here to view code image](#)

```
def _traverse_dict(self, instance_dict):
    output = {}
    for key, value in instance_dict.items():
        output[key] = self._traverse(key, value)
    return output

def _traverse(self, key, value):
    if isinstance(value, ToDictMixin):
        return value.to_dict()
    elif isinstance(value, dict):
        return self._traverse_dict(value)
    elif isinstance(value, list):
        return [self._traverse(key, i) for i in value]
    elif hasattr(value, '__dict__'):
        return self._traverse_dict(value.__dict__)
    else:
        return value
```

Here, I define an example class that uses the mix-in to make a dictionary representation of a binary tree:

[Click here to view code image](#)

```
class BinaryTree(ToDictMixin):
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right
```

Translating a large number of related Python objects into a dictionary becomes easy.

[Click here to view code image](#)

```
tree = BinaryTree(10,
    left=BinaryTree(7, right=BinaryTree(9)),
    right=BinaryTree(13, left=BinaryTree(11)))
print(tree.to_dict())

>>>
{'left': {'left': None,
          'right': {'left': None, 'right': None, 'value': 9},
          'value': 7},
 'right': {'left': {'left': None, 'right': None, 'value': 11},
           'right': None,
           'value': 13},
 'value': 10}
```

The best part about mix-ins is that you can make their generic functionality pluggable so behaviors can be overridden when required. For example, here I define a subclass of `BinaryTree` that holds a reference to its parent. This circular reference would cause the default implementation of `ToDictMixin.to_dict` to loop forever.

[Click here to view code image](#)

```
class BinaryTreeWithParent(BinaryTree):
    def __init__(self, value, left=None,
                 right=None, parent=None):
        super().__init__(value, left=left, right=right)
        self.parent = parent
```

The solution is to override the `ToDictMixin._traverse` method in the `BinaryTreeWithParent` class to only process values that matter, preventing cycles encountered by the mix-in. Here, I override the `_traverse` method to not traverse the parent and just insert its numerical value:

[Click here to view code image](#)

```
def _traverse(self, key, value):
    if (isinstance(value, BinaryTreeWithParent) and
        key == 'parent'):
        return value.value # Prevent cycles
    else:
        return super()._traverse(key, value)
```

Calling `BinaryTreeWithParent.to_dict` will work without issue because the circular referencing properties aren't followed.

[Click here to view code image](#)

```
root = BinaryTreeWithParent(10)
root.left = BinaryTreeWithParent(7, parent=root)
root.left.right = BinaryTreeWithParent(9, parent=root.left)
print(root.to_dict())
```

```
>>>
{'left': {'left': None,
          'parent': 10,
          'right': {'left': None,
                    'parent': 7,
                    'right': None,
                    'value': 9},
          'value': 7},
 'parent': None,
 'right': None,
 'value': 10}
```

By defining `BinaryTreeWithParent._traverse`, I've also enabled any class that has an attribute of type `BinaryTreeWithParent` to automatically work with `ToDictMixin`.

[Click here to view code image](#)

```
class NamedSubTree(ToDictMixin):
    def __init__(self, name, tree_with_parent):
        self.name = name
        self.tree_with_parent = tree_with_parent

my_tree = NamedSubTree('foobar', root.left.right)
print(my_tree.to_dict()) # No infinite loop

>>>
{'name': 'foobar',
 'tree_with_parent': {'left': None,
                      'parent': 7,
                      'right': None,
                      'value': 9}}
```

Mix-ins can also be composed together. For example, say you want a mix-in that provides generic JSON serialization for any class. You can do this by assuming that a class provides a `to_dict` method (which may or may not be provided by the `ToDictMixin` class).

[Click here to view code image](#)

```
class JsonMixin(object):
    @classmethod
    def from_json(cls, data):
        kwargs = json.loads(data)
        return cls(**kwargs)

    def to_json(self):
        return json.dumps(self.to_dict())
```

Note how the `JsonMixin` class defines both instance methods and class methods. Mix-ins let you add either kind of behavior. In this example, the only requirements of the `JsonMixin` are that the class has a `to_dict` method and its `__init__` method takes keyword arguments (see [Item 19: “Provide Optional Behavior with Keyword Arguments”](#)).

This mix-in makes it simple to create hierarchies of utility classes that can be serialized to and from JSON with little boilerplate. For example, here I have a hierarchy of data classes representing parts of a datacenter topology:

[Click here to view code image](#)

```
class DatacenterRack(ToDictMixin, JsonMixin):
    def __init__(self, switch=None, machines=None):
        self.switch = Switch(**switch)
        self.machines = [
            Machine(**kwargs) for kwargs in machines]

class Switch(ToDictMixin, JsonMixin):
    # ...

class Machine(ToDictMixin, JsonMixin):
    # ...
```

Serializing these classes to and from JSON is simple. Here, I verify that the data is able to be sent round-trip through serializing and deserializing:

[Click here to view code image](#)

```
serialized = """{
    "switch": {"ports": 5, "speed": 1e9},
    "machines": [
        {"cores": 8, "ram": 32e9, "disk": 5e12},
        {"cores": 4, "ram": 16e9, "disk": 1e12},
        {"cores": 2, "ram": 4e9, "disk": 500e9}
    ]
}"""

deserialized = DatacenterRack.from_json(serialized)
roundtrip = deserialized.to_json()
assert json.loads(serialized) == json.loads(roundtrip)
```

When you use mix-ins like this, it's also fine if the class already inherits from `JsonMixin` higher up in the object hierarchy. The resulting class will behave the same way.

## Things to Remember

- ◆ Avoid using multiple inheritance if mix-in classes can achieve the same outcome.
- ◆ Use pluggable behaviors at the instance level to provide per-class customization when mix-in classes may require it.
- ◆ Compose mix-ins to create complex functionality from simple behaviors.

## Item 27: Prefer Public Attributes Over Private Ones

In Python, there are only two types of attribute visibility for a class's attributes: *public* and *private*.

[Click here to view code image](#)

```
class MyObject(object):
    def __init__(self):
        self.public_field = 5
        self.__private_field = 10

    def get_private_field(self):
        return self.__private_field
```

Public attributes can be accessed by anyone using the dot operator on the object.

[Click here to view code image](#)

```
foo = MyObject()  
assert foo.public_field == 5
```

Private fields are specified by prefixing an attribute's name with a double underscore. They can be accessed directly by methods of the containing class.

[Click here to view code image](#)

```
assert foo.get_private_field() == 10
```

Directly accessing private fields from outside the class raises an exception.

[Click here to view code image](#)

```
foo.__private_field  
  
>>>  
AttributeError: 'MyObject' object has no attribute '__private_field'
```

Class methods also have access to private attributes because they are declared within the surrounding class block.

[Click here to view code image](#)

```
class MyOtherObject(object):  
    def __init__(self):  
        self.__private_field = 71  
  
    @classmethod  
    def get_private_field_of_instance(cls, instance):  
        return instance.__private_field  
  
bar = MyOtherObject()  
assert MyOtherObject.get_private_field_of_instance(bar) == 71
```

As you'd expect with private fields, a subclass can't access its parent class's private fields.

[Click here to view code image](#)

```
class MyParentObject(object):  
    def __init__(self):  
        self.__private_field = 71  
  
class MyChildObject(MyParentObject):  
    def get_private_field(self):  
        return self.__private_field  
  
baz = MyChildObject()  
baz.get_private_field()  
  
>>>  
AttributeError: 'MyChildObject' object has no attribute  
'_MyChildObject__private_field'
```

The private attribute behavior is implemented with a simple transformation of the attribute name. When the Python compiler sees private attribute access in methods like `MyChildObject.get_private_field`, it translates `__private_field` to access `_MyChildObject__private_field` instead. In this example, `__private_field` was only defined in `MyParentObject.__init__`, meaning

the private attribute's real name is `_MyParentObject__private_field`. Accessing the parent's private attribute from the child class fails simply because the transformed attribute name doesn't match.

Knowing this scheme, you can easily access the private attributes of any class, from a subclass or externally, without asking for permission.

[Click here to view code image](#)

```
assert baz._MyParentObject__private_field == 71
```

If you look in the object's attribute dictionary, you'll see that private attributes are actually stored with the names as they appear after the transformation.

[Click here to view code image](#)

```
print(baz.__dict__)

>>>
{'_MyParentObject__private_field': 71}
```

Why doesn't the syntax for private attributes actually enforce strict visibility? The simplest answer is one often-quoted motto of Python: "We are all consenting adults here." Python programmers believe that the benefits of being open outweigh the downsides of being closed.

Beyond that, having the ability to hook language features like attribute access (see [Item 32: "Use `getattr`, `getattr`, and `setattr` for Lazy Attributes](#)") enables you to mess around with the internals of objects whenever you wish. If you can do that, what is the value of Python trying to prevent private attribute access otherwise?

To minimize the damage of accessing internals unknowingly, Python programmers follow a naming convention defined in the style guide (see [Item 2: "Follow the PEP 8 Style Guide"](#)). Fields prefixed by a single underscore (like `_protected_field`) are *protected*, meaning external users of the class should proceed with caution.

However, many programmers who are new to Python use private fields to indicate an internal API that shouldn't be accessed by subclasses or externally.

[Click here to view code image](#)

```
class MyClass(object):
    def __init__(self, value):
        self.__value = value

    def get_value(self):
        return str(self.__value)

foo = MyClass(5)
assert foo.get_value() == '5'
```

This is the wrong approach. Inevitably someone, including you, will want to subclass your class to add new behavior or to work around deficiencies in existing methods (like above, how `MyClass.get_value` always returns a string). By choosing private attributes, you're only making subclass overrides and extensions cumbersome and brittle. Your potential subclassers will still access the private fields when they absolutely need to do so.

[Click here to view code image](#)

```
class MyIntegerSubclass(MyClass):
    def get_value(self):
        return int(self._MyClass__value)

foo = MyIntegerSubclass(5)
assert foo.get_value() == 5
```

But if the class hierarchy changes beneath you, these classes will break because the private references are no longer valid. Here, the `MyIntegerSubclass` class's immediate parent, `MyClass`, has had another parent class added called `MyBaseClass`:

[Click here to view code image](#)

```
class MyBaseClass(object):
    def __init__(self, value):
        self.__value = value
    # ...

class MyClass(MyBaseClass):
    # ...

class MyIntegerSubclass(MyClass):
    def get_value(self):
        return int(self._MyClass__value)
```

The `__value` attribute is now assigned in the `MyBaseClass` parent class, not the `MyClass` parent. That causes the private variable reference `self._MyClass__value` to break in `MyIntegerSubclass`.

[Click here to view code image](#)

```
foo = MyIntegerSubclass(5)
foo.get_value()

>>>
AttributeError: 'MyIntegerSubclass' object has no attribute '_MyClass__value'
```

In general, it's better to err on the side of allowing subclasses to do more by using protected attributes. Document each protected field and explain which are internal APIs available to subclasses and which should be left alone entirely. This is as much advice to other programmers as it is guidance for your future self on how to extend your own code safely.

[Click here to view code image](#)

```
class MyClass(object):
    def __init__(self, value):
        # This stores the user-supplied value for the object.
        # It should be coercible to a string. Once assigned for
        # the object it should be treated as immutable.
        self._value = value
```

The only time to seriously consider using private attributes is when you're worried about naming conflicts with subclasses. This problem occurs when a child class unwittingly defines an attribute that was already defined by its parent class.

[Click here to view code image](#)

```
class ApiClass(object):
```

```

def __init__(self):
    self._value = 5

def get(self):
    return self._value

class Child(ApiClass):
    def __init__(self):
        super().__init__()
        self._value = 'hello' # Conflicts

a = Child()
print(a.get(), 'and', a._value, 'should be different')

>>>
hello and hello should be different

```

This is primarily a concern with classes that are part of a public API; the subclasses are out of your control, so you can't refactor to fix the problem. Such a conflict is especially possible with attribute names that are very common (like `value`). To reduce the risk of this happening, you can use a private attribute in the parent class to ensure that there are no attribute names that overlap with child classes.

[Click here to view code image](#)

```

class ApiClass(object):
    def __init__(self):
        self.__value = 5

    def get(self):
        return self.__value

class Child(ApiClass):
    def __init__(self):
        super().__init__()
        self._value = 'hello' # OK!

a = Child()
print(a.get(), 'and', a._value, 'are different')

>>>
5 and hello are different

```

## Things to Remember

- ◆ Private attributes aren't rigorously enforced by the Python compiler.
- ◆ Plan from the beginning to allow subclasses to do more with your internal APIs and attributes instead of locking them out by default.
- ◆ Use documentation of protected fields to guide subclasses instead of trying to force access control with private attributes.
- ◆ Only consider using private attributes to avoid naming conflicts with subclasses that are out of your control.

## Item 28: Inherit from `collections.abc` for Custom Container Types

Much of programming in Python is defining classes that contain data and describing how such objects relate to each other. Every Python class is a container of some kind, encapsulating attributes and functionality together. Python also provides built-in container types for managing data: lists, tuples, sets, and dictionaries.

When you're designing classes for simple use cases like sequences, it's natural that you'd want to subclass Python's built-in `list` type directly. For example, say you want to create your own custom list type that has additional methods for counting the frequency of its members.

[Click here to view code image](#)

```
class FrequencyList(list):
    def __init__(self, members):
        super().__init__(members)

    def frequency(self):
        counts = {}
        for item in self:
            counts.setdefault(item, 0)
            counts[item] += 1
        return counts
```

By subclassing `list`, you get all of `list`'s standard functionality and preserve the semantics familiar to all Python programmers. Your additional methods can add any custom behaviors you need.

[Click here to view code image](#)

```
foo = FrequencyList(['a', 'b', 'a', 'c', 'b', 'a', 'd'])
print('Length is', len(foo))
foo.pop()
print('After pop:', repr(foo))
print('Frequency:', foo.frequency())

>>>
Length is 7
After pop: ['a', 'b', 'a', 'c', 'b', 'a']
Frequency: {'a': 3, 'c': 1, 'b': 2}
```

Now, imagine you want to provide an object that feels like a `list`, allowing indexing, but isn't a `list` subclass. For example, say you want to provide sequence semantics (like `list` or `tuple`) for a binary tree class.

[Click here to view code image](#)

```
class BinaryNode(object):
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right
```

How do you make this act like a sequence type? Python implements its container behaviors with instance methods that have special names. When you access a sequence item by index:

```
bar = [1, 2, 3]
bar[0]
```

it will be interpreted as:

```
bar.__getitem__(0)
```

To make the `BinaryNode` class act like a sequence, you can provide a custom implementation of `__getitem__` that traverses the object tree depth first.

[Click here to view code image](#)

```
class IndexableNode(BinaryNode):
    def _search(self, count, index):
        # ...
        # Returns (found, count)

    def __getitem__(self, index):
        found, _ = self._search(0, index)
        if not found:
            raise IndexError('Index out of range')
        return found.value
```

You can construct your binary tree as usual.

[Click here to view code image](#)

```
tree = IndexableNode(
    10,
    left=IndexableNode(
        5,
        left=IndexableNode(2),
        right=IndexableNode(
            6, right=IndexableNode(7))),
    right=IndexableNode(
        15, left=IndexableNode(11)))
```

But you can also access it like a `list` in addition to tree traversal.

[Click here to view code image](#)

```
print('LRR =', tree.left.right.right.value)
print('Index 0 =', tree[0])
print('Index 1 =', tree[1])
print('11 in the tree?', 11 in tree)
print('17 in the tree?', 17 in tree)
print('Tree is', list(tree))
```

```
>>>
LRR = 7
Index 0 = 2
Index 1 = 5
11 in the tree? True
17 in the tree? False
Tree is [2, 5, 6, 7, 10, 11, 15]
```

The problem is that implementing `__getitem__` isn't enough to provide all of the sequence semantics you'd expect.

[Click here to view code image](#)

```
len(tree)
```

```
>>>
```

TypeError: object of type 'IndexableNode' has no len()

The `len` built-in function requires another special method named `__len__` that must have an implementation for your custom sequence type.

[Click here to view code image](#)

```
class SequenceNode(IndexableNode):
    def __len__(self):
        _, count = self._search(0, None)
        return count

tree = SequenceNode(
    # ...
)

print('Tree has %d nodes' % len(tree))

>>>
Tree has 7 nodes
```

Unfortunately, this still isn't enough. Also missing are the `count` and `index` methods that a Python programmer would expect to see on a sequence like `list` or `tuple`. Defining your own container types is much harder than it looks.

To avoid this difficulty throughout the Python universe, the built-in `collections.abc` module defines a set of abstract base classes that provide all of the typical methods for each container type. When you subclass from these abstract base classes and forget to implement required methods, the module will tell you something is wrong.

[Click here to view code image](#)

```
from collections.abc import Sequence

class BadType(Sequence):
    pass

foo = BadType()

>>>
TypeError: Can't instantiate abstract class BadType with abstract methods
__getitem__, __len__
```

When you do implement all of the methods required by an abstract base class, as I did above with `SequenceNode`, it will provide all of the additional methods like `index` and `count` for free.

[Click here to view code image](#)

```
class BetterNode(SequenceNode, Sequence):
    pass

tree = BetterNode(
    # ...
)

print('Index of 7 is', tree.index(7))
print('Count of 10 is', tree.count(10))

>>>
```

```
Index of 7 is 3  
Count of 10 is 1
```

The benefit of using these abstract base classes is even greater for more complex types like `Set` and `MutableMapping`, which have a large number of special methods that need to be implemented to match Python conventions.

## Things to Remember

- ◆ Inherit directly from Python's container types (like `list` or `dict`) for simple use cases.
- ◆ Beware of the large number of methods required to implement custom container types correctly.
- ◆ Have your custom container types inherit from the interfaces defined in `collections.abc` to ensure that your classes match required interfaces and behaviors.